Documento de Requisitos — Equipe 4 (Logística)

Plataforma de Delivery · Integração de Sistemas · Versão 1.0 (rascunho) · 09/11/2026

1. Introdução

1.1 Objetivo

Este documento descreve os requisitos dos três serviços sob responsabilidade da Equipe 4 (Logística) na Plataforma de Delivery: **Frota**, **Tracking** e **Despacho**. Ele serve de referência para o desenvolvimento, os testes e a integração com as demais equipes.

1.2 Escopo

O sistema da turma é 100% back-end e formado por 15 microsserviços independentes, distribuídos entre 5 equipes. A Logística reage à aprovação de um pedido (gerado pela Equipe 3) para alocar um entregador, calcular o frete e rastrear a rota da entrega. O foco está em operações CRUD simples e, principalmente, nos contratos de comunicação entre serviços.

1.3 Definições

Termo	Significado
Entregador	Pessoa cadastrada na Frota que realiza entregas
Despacho	Registro que vincula um pedido a um entregador, com frete e status
Rastreamento	Sequência de localizações (latitude e longitude) de uma entrega
Mock	Versão simulada de uma API, gerada a partir do contrato Swagger
RBAC	Controle de acesso baseado em perfis, fornecido pela Equipe 1

2. Posição na arquitetura

A Logística está na **Onda 3 (A Reação)**: sua lógica só é ativada depois que um pedido é gerado com sucesso pela Equipe 3. Os serviços podem ser desenvolvidos com mocks de Pedidos, Identidade e Catálogo, a partir dos contratos Swagger publicados por essas equipes.

Equipe	Relação com a Logística
1. Identidade	Valida tokens JWT e permissões (RBAC) em todas as rotas protegidas
2. Catálogo	Pode fornecer dados do restaurante (por exemplo, endereço de origem da coleta)
3. Checkout	Aciona o Despacho quando o pedido é aprovado e consulta o status da entrega
5. Financeiro e Pós	Pode consumir o status da entrega para notificar o cliente

3. Convenções globais (alinhamento da turma)

Pontos que a turma inteira deve padronizar antes de iniciar o código. Os valores abaixo são a proposta da Equipe 4 e precisam ser confirmados na reunião de alinhamento.

- **Idioma:** rotas e campos JSON em português (por exemplo, `/entregadores`, `pedidoId`). A mistura de idiomas entre equipes causa falhas de integração.
- **Erros:** todas as APIs devolvem `{"erro": true, "mensagem": "Detalhes da falha"}` com o código HTTP correto (400, 401, 403, 404, 500, 503).
- **Datas e horas:** exclusivamente ISO 8601 (por exemplo, `2026-11-24T14:30:00Z`).
- **Valores financeiros:** números inteiros em centavos (R$ 25,90 trafega como `2590`).

4. Requisitos funcionais

4.1 Serviço de Frota (cadastro de entregadores)

ID	Requisito
RF-FRO-01	O sistema deve permitir cadastrar um entregador com nome, telefone, tipo de veículo e placa (quando houver).

ID	Requisito
RF-FRO-02	O sistema deve listar entregadores, com filtro por status (disponível, ocupado, inativo).
RF-FRO-03	O sistema deve consultar um entregador pelo identificador.
RF-FRO-04	O sistema deve atualizar os dados cadastrais de um entregador.
RF-FRO-05	O sistema deve alterar o status de um entregador (disponível, ocupado, inativo).
RF-FRO-06	O sistema deve permitir remover ou inativar um entregador, sem apagar o histórico de despachos já realizados.
RF-FRO-07	O sistema deve impedir o cadastro duplicado de entregadores (mesmo telefone ou mesmo documento, conforme definido).

4.2 Serviço de Tracking (rastreamento)

ID	Requisito
RF-TRA-01	O sistema deve registrar a localização (latitude, longitude e horário) de uma entrega em andamento.
RF-TRA-02	O sistema deve retornar a última localização conhecida de uma entrega.
RF-TRA-03	O sistema deve retornar o histórico de localizações de uma entrega, em ordem cronológica.
RF-TRA-04	O sistema deve validar as coordenadas recebidas (latitude entre -90 e 90, longitude entre -180 e 180).
RF-TRA-05	O sistema deve aceitar registros de localização somente para entregas existentes e ativas, consultando o serviço de Despacho.
RF-TRA-06	O sistema deve permitir registrar eventos de status da entrega (coletada, em rota, entregue) vinculados ao rastreamento.

4.3 Serviço de Despacho (frete e alocação)

ID	Requisito
RF-DES-01	O sistema deve calcular o valor do frete a partir da origem e do destino informados, devolvendo o valor em centavos e a distância estimada.
RF-DES-02	O sistema deve criar um despacho para um pedido aprovado, alocando um entregador disponível.
RF-DES-03	O sistema deve consultar a Frota por HTTP para escolher o entregador disponível e marcá-lo como ocupado.
RF-DES-04	O sistema deve devolver erro amigável quando não houver entregador disponível, sem derrubar o processo.
RF-DES-05	O sistema deve consultar um despacho pelo identificador ou pelo identificador do pedido.
RF-DES-06	O sistema deve atualizar o status do despacho (pendente, alocado, coletado, em rota, entregue, cancelado).
RF-DES-07	O sistema deve cancelar um despacho e liberar o entregador (status volta para disponível).
RF-DES-08	O sistema deve ser idempotente na criação: receber duas vezes o mesmo pedido não pode gerar dois despachos.
RF-DES-09	O sistema deve liberar o entregador automaticamente quando a entrega for concluída.

5. Endpoints propostos

Os caminhos abaixo são uma proposta inicial e devem ser formalizados no contrato OpenAPI de cada serviço.

5.1 Frota

Método	Rota	Descrição
POST	/entregadores	Cadastra entregador

Método	Rota	Descrição
GET	/entregadores	Lista entregadores (filtro opcional `?status=`)
GET	/entregadores/{id}	Consulta entregador
PUT	/entregadores/{id}	Atualiza dados
PATCH	/entregadores/{id}/status	Altera status
DELETE	/entregadores/{id}	Remove ou inativa

5.2 Tracking

Método	Rota	Descrição
POST	/entregas/{despachoId}/localizacoes	Registra uma localização
GET	/entregas/{despachoId}/localizacao	Última localização
GET	/entregas/{despachoId}/rastreamento	Histórico de localizações

5.3 Despacho

Método	Rota	Descrição
POST	/despachos/frete	Calcula o frete (sem criar despacho)
POST	/despachos	Cria despacho e aloca entregador
GET	/despachos/{id}	Consulta despacho
GET	/despachos?pedidoId=	Consulta despacho por pedido
PATCH	/despachos/{id}/status	Atualiza status
PATCH	/despachos/{id}/cancelar	Cancela e libera entregador

Todos os serviços devem expor também uma rota de saúde (`GET /saude`) para facilitar a verificação durante a demonstração.

6. Modelo de dados (resumo)

Cada serviço mantém seu próprio banco de dados em container, sem acesso ao banco de outras equipes.

Entregador (Frota)

Campo	Tipo	Observação
id	texto ou inteiro	Identificador
nome	texto	Obrigatório
telefone	texto	Obrigatório, único
veiculo	texto	Moto, bicicleta, carro etc.
placa	texto	Opcional
status	texto	disponivel, ocupado, inativo
criadoEm	data e hora ISO 8601	

Despacho (Despacho)

Campo	Tipo	Observação
id	texto ou inteiro	Identificador
pedidoId	texto ou inteiro	Vem da Equipe 3, único
entregadorId	texto ou inteiro	Vem da Frota
origem	latitude e longitude	Local da coleta
destino	latitude e longitude	Local da entrega
distanciaMetros	inteiro	Estimada
freteCentavos	inteiro	Valor em centavos
status	texto	pendente, alocado, coletado, em_rota, entregue, cancelado
criadoEm / atualizadoEm	data e hora ISO 8601	

Localização (Tracking)

Campo	Tipo	Observação
id	texto ou inteiro	Identificador

Campo	Tipo	Observação
despachoId	texto ou inteiro	Referência ao despacho
latitude / longitude	decimal	Coordenadas
registradoEm	data e hora ISO 8601	

7. Integrações

7.1 Serviços consumidos pela Logística

Serviço	Equipe	Uso
Autenticação e Autorização	1	Validar o token JWT e as permissões de cada requisição
Restaurantes	2	Obter o endereço ou as coordenadas de origem, se necessário
Pedidos	3	Confirmar que o pedido existe e está aprovado
Frota (interno)	4	O Despacho consulta e atualiza o status do entregador
Despacho (interno)	4	O Tracking valida a existência e o estado do despacho

7.2 Serviços que consomem a Logística

Serviço	Equipe	Uso
Pedidos	3	Aciona o Despacho e consulta o status da entrega
Notificações	5	Pode avisar o cliente conforme o status do rastreamento

7.3 Regras de integração

- A comunicação entre serviços é feita somente por chamadas HTTP, nunca por acesso direto ao banco de outro grupo.
- Cada chamada externa deve ter tempo limite (timeout) e tratamento de exceção.

- Se um serviço dependente estiver fora do ar, a Logística responde com erro amigável (por exemplo, 503 com a mensagem padronizada) e o processo continua ativo.

8. Regras de negócio

- **RN-01:** somente entregadores com status `disponivel` podem ser alocados.
- **RN-02:** ao alocar, o entregador passa a `ocupado`; ao concluir ou cancelar o despacho, volta a `disponivel`.
- **RN-03:** cada pedido possui no máximo um despacho ativo.
- **RN-04:** o frete é calculado em centavos, a partir da distância entre origem e destino (por exemplo, fórmula de Haversine), com valor base e valor por quilômetro configuráveis por variáveis de ambiente. A fórmula final deve ser validada pela equipe.
- **RN-05:** a transição de status do despacho segue a ordem: pendente, alocado, coletado, em rota, entregue. O cancelamento é permitido antes da entrega.
- **RN-06:** localizações só podem ser registradas para despachos em andamento (coletado ou em rota).
- **RN-07:** entregadores inativos não podem ser alocados nem receber novas entregas.

9. Requisitos não funcionais

ID	Requisito
RNF-01	**Documentação:** todos os contratos devem estar documentados em Swagger/OpenAPI, permitindo que outras equipes usem mocks antes de a API real estar pronta.
RNF-02	**Conteinerização:** cada serviço deve ter `Dockerfile` e `docker-compose.yml`, de modo que outra equipe execute o serviço com um único comando e sem instalar dependências locais.
RNF-03	**Banco isolado:** cada serviço sobe seu próprio container de banco de dados; é proibido cruzar dados no banco de outro grupo.
RNF-04	**Testes:** testes automatizados unitários e de integração, com cobertura mínima de 75%.
RNF-05	**Resiliência:** falha em um serviço dependente não pode derrubar o processo; a exceção é tratada e a API devolve erro amigável.

ID	Requisito
RNF-06	**Logs:** as chamadas HTTP entre serviços devem aparecer no console do container (por exemplo, "Despacho consultando API de Frota"), pois é a prova de integração na demonstração.
RNF-07	**Segurança:** rotas protegidas por token JWT, com permissões validadas junto à Equipe 1.
RNF-08	**Execução paralela:** cada serviço roda em porta própria, sem compartilhar memória com os demais.
RNF-09	**Configuração:** URLs de outros serviços, portas e parâmetros de frete definidos por variáveis de ambiente.
RNF-10	**Padronização:** respostas de erro, datas e valores financeiros seguem as convenções globais da seção 3.

10. Fluxo de demonstração

O professor executará o fluxo pelo Postman ou Insomnia. O trecho da Logística acontece após o pedido ser gerado:

1. O cliente faz login (Equipe 1) e busca um prato (Equipe 2).
2. O pedido é criado e aprovado (Equipe 3).
3. Pedidos chama `POST /despachos` na Logística.
4. O Despacho consulta a Frota, escolhe um entregador disponível, calcula o frete e registra o despacho.
5. O entregador envia localizações ao Tracking durante a rota.
6. O status é atualizado até `entregue`, e o entregador volta a ficar disponível.
7. Os logs de cada container evidenciam as chamadas entre os serviços.

11. Decisões em aberto

Itens a definir com a turma e com a equipe antes do desenvolvimento:

- Idioma definitivo das rotas e dos campos JSON.
- Como o Despacho é acionado: chamada HTTP direta de Pedidos ou webhook.
- Formato final de identificadores (inteiro ou UUID) e de coordenadas.

- Lista oficial de status de entrega, compartilhada com Pedidos e Notificações.
- Fonte do endereço de origem (Catálogo ou enviado por Pedidos) e do destino (Pedidos ou Perfis).
- Fórmula e parâmetros do cálculo de frete.
- Linguagem, framework, banco de dados e portas de cada serviço.

12. Entregáveis e critérios de aceite

Entregável	Critério de aceite
Três serviços funcionando	Endpoints da seção 5 respondendo conforme o contrato
Contratos OpenAPI	Swagger publicado para cada serviço, utilizável para gerar mocks
Docker	Cada serviço sobe com um único comando, junto com seu banco
Testes	Cobertura igual ou superior a 75% em cada serviço
Integração	Fluxo Login, Buscar prato, Fazer pedido, Despacho executado com sucesso no Postman ou Insomnia
Resiliência	Com um serviço dependente parado, a API responde erro amigável e continua ativa
Logs	Console mostra as chamadas entre serviços durante a demonstração